
Software Requirements Specification

for

Kottayam Online Judge - KOJ

Version 1.0 approved

Prepared by Prajwal, Chandra Teja, Shivkarthik, Hari Krishnan S

Group 3 IIIT Kottayam

4th August, 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References.....	2
2. Overall Description	3
2.1 Product Perspective.....	3
2.2 Product Functions	3
2.3 User Classes and Characteristics	4
2.4 Operating Environment.....	4
2.5 Design and Implementation Constraints	5
2.6 User Documentation	5
2.7 Assumptions and Dependencies	5
3. External Interface Requirements	6
3.1 User Interfaces	6
3.2 Hardware Interfaces	6
3.3 Software Interfaces	6
3.4 Communications Interfaces	7
4. System Features	7
4.1 Authentication and User Management.....	7
4.2 Problem Management	8
5. Other Nonfunctional Requirements	12
5.1 Performance Requirements	12
5.2 Safety Requirements	13
5.3 Security Requirements	13
5.4 Software Quality Attributes	13
5.5 Business Rules	14
6. Other Requirements	14
Appendix A: Glossary.....	15
Appendix B: To Be Determined List	15

Revision History

Author	Date	Changes	Version
Group 3	4 Aug 2026	Initial version	1.0

1. Introduction

1.1 Purpose

This document defines the software requirements for Kottayam Online Judge (KOJ), version 1.0. KOJ is a web-based competitive programming platform that supports problem hosting, code submission, automated judging, and contest management. It is designed for use by IIT Kottayam students and faculty for conducting programming contests and practice sessions.

This SRS covers the complete system including the web frontend, the backend API, the judge engine, and the database.

1.2 Document Conventions

- Requirements are identified with unique IDs in the format `REQ-<CATEGORY>-<NUMBER>` (e.g., REQ-AUTH-01).
- Priority levels: **High** (must have for v1), **Medium** (should have, may defer to Phase 2), **Low** (nice to have).
- This document follows the IEEE 830-1998 standard for Software Requirements Specifications.
- All timestamps use IST unless otherwise stated.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- **Developers** (primary audience): Read Sections 3 and 4 in full. These contain the interface and feature-level requirements you will implement.
- **Project managers**: Read Sections 1.4, 2.2, and 5.1 for scope, feature summary, and performance targets.
- **Course evaluator**: Read Sections 1.4, 2.1, 2.5, and Appendix B for product scope, architectural decisions, and design rationale.
- **Problem setters and admins**: Read Section 2.3 and Section 4.2 for your role in the system.

Recommended reading order: start with Section 1.4 (Product Scope) for context, then Section 2 (Overall Description), then the relevant parts of Section 4 based on your role.

1.4 Product Scope

What KOJ is:

- A web application where problem setters upload problems with test cases, and contestants submit code solutions.
- A code judge that compiles and executes submissions in an isolated environment, compares output against expected results, and returns verdicts (Accepted, Wrong Answer, Time Limit Exceeded, etc.).
- A contest platform that hosts timed competitions with live leaderboards and penalty-based scoring.

What KOJ is not (v1 scope):

- It is not a cloud-based IDE or code editor.
- It does not support interactive problems (planned for v2).
- It does not provide full OS-level sandboxing like Docker (uses process-level resource limits, sufficient for a trusted college user base).

Benefits:

- Gives IIIT Kottayam a self-hosted platform for programming contests instead of relying on external platforms that may be unavailable or unsuitable.
- Provides hands-on experience building a system with real distributed components, a code execution pipeline, and real-time UI.
- Demonstrates clear architectural patterns: Layered, Pipe-and-Filter, Event-Driven, and Master-Slave.

1.5 References

1. IEEE 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. FastAPI Documentation. <https://fastapi.tiangolo.com/>
3. Next.js Documentation. <https://nextjs.org/docs>
4. Neon Documentation. <https://neon.tech/docs>
5. Clerk Documentation. <https://clerk.com/docs>
6. testlib.h - Codeforces Checker Library. <https://github.com/MikeMirzayanov/testlib>
7. Python subprocess Module. <https://docs.python.org/3/library/subprocess.html>
8. Azure App Service Documentation. <https://learn.microsoft.com/en-us/azure/app-service/>
9. DOMjudge - ACM ICPC Judge System. <https://www.domjudge.org/>
10. SPOJ - Sphere Online Judge. <https://www.spoj.com/>

2. Overall Description

2.1 Product Perspective

KOJ is a self-contained web application built as a new product, not a replacement for or extension of an existing system. It is designed as a three-tier architecture with clear separation between the presentation layer, application logic layer, and data layer.

The frontend communicates with the backend through REST API calls and WebSocket connections. Authentication is handled by Clerk, which manages user sessions, OAuth providers (Google, GitHub), and JWT token verification. The backend handles all logic, dispatches code judging to an isolated subprocess, and reads/writes data to Neon (PostgreSQL). The judge engine runs as a subprocess spawned by the backend, with no direct access to the database or the frontend. Realtime verdict and leaderboard updates are delivered via FastAPI WebSocket connections.

This separation is deliberate. The judge module is loosely coupled from the web layer. The API does not know how the judge works internally; it only calls a function that returns a verdict. This lets us test the judge in complete isolation and replace its implementation without affecting the rest of the system.

Architectural styles used:

Style	Where in KOJ	Why it is used
Layered	Frontend -> API -> Database	Each request passes through all three layers with no skipping
Pipe-and-Filter	Judge pipeline: receive -> compile -> execute -> check -> verdict	Each stage has one job and a fixed input/output contract
Event-Driven	FastAPI WebSocket pushes verdict changes to connected clients	Backend writes verdict, pushes update to UI via WebSocket, no polling required
Master-Slave	FastAPI (master) dispatches to judge subprocess (slave)	Subprocess has no awareness of the web layer

Coupling and cohesion decisions:

- The judge module is highly cohesive (does exactly one thing: take code + input, return verdict) and loosely coupled (the web layer calls it via a clean interface and does not care how it runs code internally).
- The contest module and problem module are deliberately decoupled. A problem does not know which contest it belongs to at the problem-object level; the contest holds references to problems. This allows problems to exist independently after a contest ends (the archive feature).
- A naive design would tightly couple Contest and Problem. We rejected that because it would make post-contest practice impossible without duplicating problem data.

2.2 Product Functions

The major functions of KOJ are:

- **User Registration and Authentication:** Users sign up with email/password or OAuth (GitHub, Google). Three roles: Contestant, Problem Setter, Admin.

- **Problem Management:** Problem setters create problems, upload test cases (input/output text files), and manage problem metadata (title, time limit, memory limit, difficulty).
- **Contest Management:** Admins create contests, add problems, set start/end times, choose between open enrollment or invite-based registration.
- **Code Submission:** Contestants select a problem, write code in the browser or paste code, choose a language, and submit.
- **Automated Judging:** The system compiles (if needed), executes, and evaluates submissions against test cases. Verdicts are returned in real time.
- **Live Leaderboard:** A real-time ranked list of contestants showing problems solved and penalty time. Updates automatically as verdicts come in.
- **Practice Mode:** Problems are available outside of contests for practice. Submissions get immediate feedback.
- **Rate Limiting:** Submissions are throttled to prevent abuse (max 1 submission per 30 seconds per user).

2.3 User Classes and Characteristics

<i>User Class</i>	<i>Description</i>	<i>Technical Level</i>	<i>Frequency of Use</i>
<i>Contestant</i>	<i>Writes and submits code solutions to problems. Participates in contests.</i>	<i>Moderate (knows at least one programming language)</i>	<i>High (daily during contest season)</i>
<i>Problem Setter</i>	<i>Creates problems, writes problem statements, uploads test cases. May also participate as a contestant.</i>	<i>High (familiar with competitive programming and test case design)</i>	<i>Medium (before/during contest creation)</i>
<i>Admin</i>	<i>Manages the platform: creates contests, assigns problem setters, manages user roles, monitors system health.</i>	<i>High (technical administration)</i>	<i>Low to medium (as needed)</i>

2.4 Operating Environment

- **Frontend:** Deployed on Vercel. Requires a modern web browser (Chrome, Firefox, Safari, Edge) with JavaScript enabled.
- **Backend:** Deployed on Azure App Service (primary) or GCP Cloud Run (fallback). Runs Python 3.11+ with FastAPI.
- **Database:** Neon (managed PostgreSQL 15+ with branching support).
- **Auth:** Clerk (managed authentication service with OAuth, email/password, and JWT verification).
- **Judge Engine:** Runs on the same server as the backend. Requires Python 3.11+ and compilers for supported languages (gcc for C/C++, javac for Java).
- **Supported Judge Languages (v1):** C, C++, Python, Java (High priority); Go, Rust, JavaScript/Node.js (Medium priority, if timeline permits).

2.5 Design and Implementation Constraints

1. **Budget:** The project must run within free tiers and Azure student credits. No paid infrastructure beyond what student credits provide.
2. **Team size:** 4 developers.
3. **Sandboxing:** KOJ uses process-level isolation via Python's `resource` module (`rlimit`) instead of Docker containers. This is sufficient for a trusted college user base but not for open internet deployment. A Docker-based upgrade path is documented for v2.
4. **No interactive problems in v1:** Only problems with static input/output test cases are supported.
5. **Single time limit per problem:** No per-language time limit multipliers (Python submissions will be harder to pass at tight time limits, same as on SPOJ).
6. **Leaderboard consistency model:** The leaderboard uses a push-based model via WebSocket. The backend writes the verdict to the database and immediately pushes the update to connected clients. This gives sub-second propagation without depending on a third-party realtime service.
7. **Future Enhancement (v2):** The judge will employ container-based sandboxing (e.g., Docker) or OS-level isolation mechanisms (such as namespaces/seccomp) to enforce filesystem isolation and stronger security.

2.6 User Documentation

The following documentation will be delivered with the software:

- **README.md:** Project setup instructions, architecture overview, and contribution guidelines for developers.
- **Problem Setter Guide:** How to create problems, upload test cases, and validate them.
- **User Guide:** How to register, browse problems, submit code, and participate in contests.
- **Deployment Guide:** How to deploy the system on Azure, GCP, or locally.

2.7 Assumptions and Dependencies

Assumptions:

- The system will be used by a trusted user base (IIIT Kottayam students and faculty). Full OS-level sandboxing (Docker, seccomp) is not required for v1.
- Azure student credits will be available for deployment. If not, GCP free tier or local deployment is the fallback.
- Problem setters will validate their own test cases before publishing. Automated test case validation is not included in v1.
- All users have modern web browsers with JavaScript enabled.

- The system will handle up to 200 concurrent users during a college-level contest.

Dependencies:

- Neon must be available for database hosting.
- Clerk must be available for authentication services.
- Azure App Service or GCP Cloud Run must be available for backend hosting.
- Vercel must be available for frontend hosting.
- Supported language compilers (gcc, javac, node, go, rustc) must be installed on the backend server.

3. External Interface Requirements

3.1 User Interfaces

The system provides a web-based interface accessible through standard browsers. Key screens include:

- **Home Page:** Shows recent problems, active contests, and user stats.
- **Problem List:** Filterable list of all problems with status indicators (solved/unsolved).
- **Problem View:** Problem statement, code editor (or textarea), language selector, submit button.
- **Submission Status:** Real-time verdict display (Pending, Running, Accepted, Wrong Answer, etc.).
- **Contest Page:** Contest info, problem list for that contest, countdown timer, and leaderboard.
- **Leaderboard:** Ranked list of contestants with problems solved and penalty time. Updates in real time.
- **Admin Dashboard:** Problem management, contest creation, user role management.

The UI follows a clean, minimal design similar to Codeforces. No complex animations or heavy visual effects. Responsive design for desktop and tablet (mobile is not a priority for v1).

3.2 Hardware Interfaces

KOJ does not interface with any specialized hardware. It runs on standard cloud VMs or local machines. The backend server should have at least 2 CPU cores and 4 GB RAM to handle concurrent judging.

3.3 Software Interfaces

Interface	Purpose	Direction	Protocol
Neon SDK (psycpg2/asyncpg)	Database read/write from backend	Bidirectional	PostgreSQL wire protocol over TLS/SSL
Clerk JS SDK	Authentication and session management from frontend	Bidirectional	HTTPS
Clerk Backend SDK (Python)	JWT verification and user management from backend	Bidirectional	HTTPS

Interface	Purpose	Direction	Protocol
FastAPI	Backend API server	Inbound	HTTP/REST
FastAPI WebSocket	Realtime verdict and leaderboard updates	Bidirectional	WebSocket
Next.js	Frontend server	Inbound	HTTP
Judge subprocess	Code execution	Outbound from backend	Python subprocess (stdin/stdout)
Vercel	Frontend hosting	N/A	HTTPS
Azure App Service / GCP Cloud Run	Backend hosting	N/A	HTTPS

3.4 Communications Interfaces

Frontend to Backend: REST API over HTTPS. JSON request/response format.

- **Frontend to Backend (Realtime):** WebSocket connection for live verdict and leaderboard updates.
- **Frontend to Clerk:** HTTPS for authentication flows (login, signup, OAuth redirects).
- **Backend to Clerk:** HTTPS for JWT verification via Clerk's JWKS endpoint.
- **Backend to Judge:** Python subprocess.Popen(). The backend writes source code and test input to the subprocess stdin, reads output and exit code from stdout/stderr.
- **Backend to Neon:** PostgreSQL wire protocol over TLS/SSL for all database operations.

4. System Features

4.1 Authentication and User Management:

4.1.1 Description and Priority:

Users register and log in to the system via Clerk. Three roles are supported: Contestant (default), Problem Setter, and Admin. OAuth login via GitHub and Google is supported alongside email/password, all managed by Clerk.

Priority: **High**

4.1.2 Stimulus/Response Sequences

1. User clicks "Sign Up" and enters email, password, and display name.
2. System creates the account, assigns Contestant role, and redirects to home page.
3. User clicks "Login with GitHub" and is redirected to GitHub OAuth.
4. System authenticates via GitHub, creates account if new, and redirects to home page.
5. Admin navigates to admin panel and changes a user's role from Contestant to Problem Setter.
6. System updates the role and the user gains problem creation access on next page load.

4.1.3 Functional Requirements

Requirement	Description	Priority
REQ-AUTH-01	The system shall allow users to register with email and password.	High
REQ-AUTH-02	The system shall support OAuth login via GitHub and Google.	High

Requirement	Description	Priority
REQ-AUTH-03	The system shall assign the Contestant role to new users by default.	High
REQ-AUTH-04	The system shall allow admins to change user roles (Contestant, Problem Setter, Admin).	High
REQ-AUTH-05	The system shall maintain session state using Clerk (JWT tokens verified via JWKS).	High
REQ-AUTH-06	The system shall reject registration with duplicate email addresses and display an appropriate error message.	High
REQ-AUTH-07	The system shall allow users to reset their password via email.	Medium

4.2 Problem Management

4.2.1 Description and Priority

Problem setters create problems with statements, metadata, and test cases. Test cases are uploaded as text files (input/output pairs). Problems exist in three states: Draft, Contest-Active, and Published.

Priority: **High**

4.2.2 Stimulus/Response Sequences

1. Problem Setter clicks "Create Problem" and fills in title, statement (markdown), time limit, memory limit, and difficulty.
2. System creates the problem in Draft state.
3. Problem Setter uploads test case files (input.txt, output.txt pairs).
4. System stores test cases and displays a preview.
5. Admin adds the problem to a contest.
6. System transitions the problem to Contest-Active state.
7. Contest ends.
8. System transitions the problem to Published state (available for practice).

4.2.3 Functional Requirements

Requirement	Description	Priority
REQ-PROB-01	The system shall allow Problem Setters to create problems with title, statement (markdown), time limit, memory limit, and difficulty level.	High
REQ-PROB-02	The system shall support uploading test cases as input/output text file pairs.	High
REQ-PROB-03	The system shall enforce a maximum test case file size of 10 MB per file.	High
REQ-PROB-04	The system shall maintain a problem state machine: Draft -> Contest-Active -> Published.	High
REQ-PROB-05	Problems in Draft state shall be visible only to the Problem Setter and Admin.	High
REQ-PROB-06	Problems in Contest-Active state shall be visible only to registered contest participants during the contest.	High
REQ-PROB-07	Problems in Published state shall be visible to all users for practice.	High
REQ-PROB-08	The system shall support markdown rendering in problem statements.	Medium

Requirement	Description	Priority
REQ-PROB-09	The system shall allow Problem Setters to edit problems in Draft state.	High
REQ-PROB-10	The system shall prevent editing of problems in Contest-Active state after the contest has started.	High

4.3 Code Submission and Judge Pipeline

4.3.1 Description and Priority

This is the core feature of KOJ. A contestant submits code, and the system compiles (if needed), executes, and evaluates it against all test cases for the problem. The judge runs as a separate subprocess, completely decoupled from the web layer. Verdicts are returned in real time.

Priority: **High**

The judge pipeline follows a pipe-and-filter architecture:

Receive -> Compile -> Execute -> Check -> Verdict

Each stage has a single responsibility:

- **Receive:** Accept source code, language, and problem ID.
- **Compile:** Translate source code to executable (skipped for interpreted languages like Python).
- **Execute:** Run the executable with test case input, enforcing time and memory limits.
- **Check:** Compare output against expected output (whitespace-normalized string comparison in v1; custom checker in v2).
- **Verdict:** Return AC (Accepted), WA (Wrong Answer), TLE (Time Limit Exceeded), MLE (Memory Limit Exceeded), RE (Runtime Error), or CE (Compilation Error).

4.3.2 Stimulus/Response Sequences

1. Contestant selects a problem, writes or pastes code, selects a language, and clicks "Submit."
2. System validates the submission (checks language is supported, code is not empty) and returns HTTP 202 Accepted with a submission ID.
3. Backend enqueues the submission for judging.
4. Judge subprocess receives the source code and test cases.
5. If the language requires compilation (C, C++, Java), the judge compiles the code. If compilation fails, verdict is CE.
6. Judge executes the compiled binary (or interprets the script) with each test case as stdin, enforcing time limit (wall-clock) and memory limit (rlimit).
7. If execution exceeds time limit, verdict is TLE. If memory limit is exceeded, verdict is MLE. If the process crashes or returns non-zero exit code, verdict is RE.
8. For each test case, judge compares output against expected output using whitespace-normalized string comparison.
9. If all test cases match, verdict is AC. If any test case fails, verdict is WA.
10. Backend writes the verdict to the database.
11. Backend pushes the verdict to the contestant via WebSocket.
12. Contestant sees the verdict update in real time without refreshing the page.

4.3.3 Functional Requirements

Requirement	Description	Priority
REQ-JUDGE-01	The system shall accept code submissions via POST request and return a submission ID.	High
REQ-JUDGE-02	The system shall support the following languages: C, C++, Python, Java (v1); Go, Rust, JavaScript (v2).	High

Requirement	Description	Priority
REQ-JUDGE-03	The system shall compile C/C++ submissions using gcc/g++ and Java submissions using javac.	High
REQ-JUDGE-04	The system shall execute submissions in a subprocess with CPU time limits enforced via <code>resource.setrlimit(RESOURCE_RLIMIT_CPU)</code> .	High
REQ-JUDGE-05	The system shall enforce wall-clock time limits via subprocess timeout.	High
REQ-JUDGE-06	The system shall enforce memory limits via <code>resource.setrlimit(RESOURCE_RLIMIT_AS)</code> .	High
REQ-JUDGE-07	The system shall limit the number of subprocesses per submission to prevent fork bombs.	High
REQ-JUDGE-08	The system shall execute submissions in an isolated working directory containing only the files required for judging.	High
REQ-JUDGE-09	The system shall compare submission output against expected output using whitespace-normalized string comparison (trailing whitespace and newlines are ignored).	High
REQ-JUDGE-10	The system shall run submissions against all test cases for a problem and return a single verdict.	High
REQ-JUDGE-11	The system shall return verdicts: AC, WA, TLE, MLE, RE, CE, or PE (Presentation Error).	High
REQ-JUDGE-12	The system shall deliver verdicts to the client in real time via WebSocket.	High
REQ-JUDGE-13	The system shall handle up to 10 concurrent judging tasks without significant performance degradation.	High
REQ-JUDGE-14	The system shall log all submission results (submission ID, verdict, execution time, memory used) to the database.	High
REQ-JUDGE-15	The system shall support custom checker programs for problems that require semantic output validation (v2).	Medium
REQ-JUDGE-16	The system shall allow problem setters to specify a custom time limit per problem (default: 2 seconds).	High
REQ-JUDGE-17	The system shall allow problem setters to specify a memory limit per problem (default: 256 MB).	High

4.4 Contest Management

4.4.1 Description and Priority

Admins create and manage contests. Each contest has a set of problems, a start time, an end time, and a registration mode (open or invite-based). Contestants register for contests and compete for ranking.

Priority: **High**

4.4.2 Stimulus/Response Sequences

1. Admin creates a contest with title, description, start time, end time, and registration mode.
2. Admin adds problems to the contest from the existing problem pool.
3. System registers the contest and makes it visible on the contest list.
4. Contestant clicks "Register" for an open contest or enters an invite code for a private contest.
5. System adds the contestant to the contest participant list.

6. Contest start time arrives. System makes contest problems visible to registered participants.
7. Contest ends. System locks submissions and makes problems visible to all users (Published state).
8. System displays the final leaderboard for the contest.

4.4.3 Functional Requirements

Requirement	Description	Priority
REQ-CONT-01	The system shall allow Admins to create contests with title, description, start time, end time, and registration mode.	High
REQ-CONT-02	The system shall support two registration modes: Open Enrollment and Invite-Based (via code).	High
REQ-CONT-03	The system shall allow Admins to add existing problems to a contest.	High
REQ-CONT-04	The system shall automatically make contest problems visible at the start time and lock them at the end time.	High
REQ-CONT-05	The system shall allow contestants to register for open contests with one click.	High
REQ-CONT-06	The system shall allow contestants to register for invite-based contests by entering a valid invite code.	High
REQ-CONT-07	The system shall prevent submissions to a contest problem after the contest end time.	High
REQ-CONT-08	The system shall display a countdown timer showing time remaining in the contest.	Medium
REQ-CONT-09	The system shall support both individual and team contest modes (team mode: single shared device, no separate accounts).	Low

4.5 Live Leaderboard

4.5.1 Description and Priority

The leaderboard shows a real-time ranking of contestants in a contest. Rankings are based on problems solved (descending) and penalty time (ascending). Penalty is calculated as: time of first accepted solution (in minutes from contest start) + 20 minutes per incorrect submission before the first AC.

Priority: **High**

4.5.2 Stimulus/Response Sequences

1. Contestant submits code during a live contest.
2. Backend judges the submission and writes the verdict to the database.
3. Backend pushes the updated leaderboard to all connected clients via WebSocket.
4. Frontend receives the update and recalculates the leaderboard.
5. All connected clients see the updated ranking without refreshing the page.

4.5.3 Functional Requirements

Requirement	Description	Priority
REQ-LB-01	The system shall calculate rankings based on problems solved (descending) and penalty time (ascending).	High
REQ-LB-02	The system shall calculate penalty as: first AC time (minutes) + (20 * wrong	High

Requirement	Description	Priority
	submissions before first AC).	
REQ-LB-03	The system shall update the leaderboard in real time using WebSocket push from the backend.	High
REQ-LB-04	The leaderboard shall update within 2 seconds of a verdict write via WebSocket push.	High
REQ-LB-05	The system shall display the leaderboard to all users during and after the contest.	High
REQ-LB-06	The system shall show per-problem status (solved/unsolved/attempted) for each contestant on the leaderboard.	Medium

4.6 Rate Limiting and Abuse Prevention

4.6.1 Description and Priority

To prevent abuse and ensure fair resource usage, the system limits how frequently users can submit code.

Priority: **Medium**

4.6.2 Functional Requirements

Requirement	Description	Priority
REQ-RATE-01	The system shall limit each user to 1 submission per 30 seconds per problem.	High
REQ-RATE-02	The system shall return HTTP 429 (Too Many Requests) with a retry-after header when rate limit is exceeded.	High
REQ-RATE-03	The system shall display a visible countdown timer on the submit button showing time until next allowed submission.	Medium

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Requirement	Description	Priority
REQ-PERF-01	The system shall accept a code submission and return a submission ID within 1 second.	High
REQ-PERF-02	The judge shall return a verdict for simple problems (under 1s execution time) within 5 seconds of submission.	High
REQ-PERF-03	The system shall support at least 50 concurrent users during a contest without response times exceeding 3 seconds.	High
REQ-PERF-04	The leaderboard shall update within 2 seconds of a verdict being written to the database.	High
REQ-PERF-05	The frontend shall load the problem list page within 2 seconds on a standard broadband connection.	Medium

Requirement	Description	Priority
REQ-PERF-06	The judge shall handle up to 10 concurrent judging tasks per server instance.	High

5.2 Safety Requirements

Requirement	Description	Priority
REQ-SAFE-01	The system shall prevent submitted code from accessing or modifying system files outside the judge sandbox.	High
REQ-SAFE-02	The system shall kill any submission process that exceeds its allocated CPU time or wall-clock time limit.	High
REQ-SAFE-03	The system shall limit the number of child processes a submission can spawn to prevent fork bombs.	High
REQ-SAFE-04	The system shall not store or expose user source code to other users without explicit permission.	Medium

5.3 Security Requirements

Requirement	Description	Priority
REQ-SEC-01	All communication between frontend and backend shall use HTTPS.	High
REQ-SEC-02	User passwords shall be stored using bcrypt hashing (handled by Clerk).	High
REQ-SEC-03	API endpoints shall validate JWT tokens for authenticated routes.	High
REQ-SEC-04	Admin-only endpoints shall verify the user has Admin role before processing the request.	High
REQ-SEC-05	The system shall implement CSRF protection for state-changing requests.	Medium
REQ-SEC-06	The system shall sanitize all user inputs to prevent SQL injection and XSS attacks.	High

5.4 Software Quality Attributes

Attribute	Description	Target
Availability	The system should be available during scheduled contest windows.	99.5% uptime during contests
Reliability	The judge shall return correct verdicts for well-formed submissions.	No incorrect AC verdicts for valid test cases
Maintainability	The codebase shall follow modular architecture with clear separation of concerns.	Each module independently testable
Testability	The judge module shall be testable without the web layer.	40+ unit tests for judge pipeline
Usability	The UI shall be intuitive for users familiar with Codeforces or SPOJ.	No tutorial needed for basic workflow
Portability	The system shall run on Azure, GCP, or a local machine with minimal configuration changes.	Docker-free deployment

5.5 Business Rules

Rule	Description
------	-------------

BR-01	Only Admins can create contests.
BR-02	Only Problem Setters and Admins can create problems.
BR-03	Only Admins can change user roles.
BR-04	A problem must be in Draft state before it can be added to a contest.
BR-05	Submissions to a contest problem are locked after the contest end time.
BR-06	A user can only register for a contest before it starts.
BR-07	The penalty for a problem is only calculated after the first AC verdict. Subsequent ACs do not change the penalty.
BR-08	Problems that are part of an active contest cannot be edited.

6. Other Requirements

1. **Database:** All persistent data shall be stored in Neon (PostgreSQL). The schema shall include tables for users, problems, test_cases, submissions, contests, contest_problems, contest_registrations, and leaderboards.
2. **Logging:** The backend shall log all judge executions (submission ID, language, verdict, execution time, memory usage) for audit and debugging purposes.
3. **Deployment:** The system shall support one-command deployment via Vercel (frontend) and Azure App Service / GCP Cloud Run (backend). Environment variables shall be configurable through each platform's dashboard.
4. **Version Control:** The project shall use Git with a feature-branch workflow. All code shall be hosted on a shared GitHub repository.
5. **Future Enhancements (v2):** Custom checker programs via testlib.h, interactive problems, Docker-based sandboxing, per-language time limit multipliers, Redis caching layer, automated test case validation.

Appendix A: Glossary

Term	Definition
AC	Accepted. The submission produced correct output for all test cases.
WA	Wrong Answer. The submission produced incorrect output for one or more test cases.
TLE	Time Limit Exceeded. The submission took longer than the allowed time limit.
MLE	Memory Limit Exceeded. The submission used more memory than allowed.
RE	Runtime Error. The submission crashed or returned a non-zero exit code.
CE	Compilation Error. The submission failed to compile.
PE	Presentation Error. The submission is logically correct but has formatting issues (e.g., extra whitespace).
Judge	The system component that compiles, executes, and evaluates code submissions.
Verdict	The result of evaluating a submission (AC, WA, TLE, etc.).
Test Case	A pair of input and expected output files used to evaluate a submission.
Penalty	Time-based scoring metric in contests: first AC time + 20 minutes per wrong attempt.
rlimit	Resource limit. A Linux kernel feature for restricting process resource usage (CPU time, memory, etc.).
Pipe-and-Filter	An architectural pattern where data flows through a sequence of processing stages.
Event-Driven	An architectural pattern where components communicate by producing and consuming events.
Master-Slave	An architectural pattern where a master component dispatches work to slave components.
OAuth	An open standard for token-based authentication and authorization.
JWT	JSON Web Token. A compact, URL-safe token for securely transmitting information between parties.
Neon	A serverless PostgreSQL platform providing managed database hosting with branching support.
Clerk	A managed authentication service providing OAuth, email/password login, session management, and JWT verification.
FastAPI	A modern Python web framework for building APIs with automatic OpenAPI documentation.
Next.js	A React framework for building server-side rendered and static web applications.

Appendix B: To Be Determined List

Item	Description	Target Resolution
TBD-01	Redis caching layer for leaderboard and problem metadata. Depends on whether timeline permits Phase 2.	September 2026
TBD-02	Custom checker programs using testlib.h. Planned for v2.	Post v1 release
TBD-03	Go, Rust, and JavaScript/Node.js language support in the judge. Medium priority, depends on timeline.	September 2026

Item	Description	Target Resolution
TBD-04	Team contest mode (low priority). Implementation depends on demand from college fest.	Post v1 release
TBD-05	Password reset via email flow. Depends on Clerk email configuration.	August 2026
TBD-06	Exact Azure/GCP deployment configuration. Depends on student credit approval status.	August 2026
TBD-07	Interactive problems support. Planned for v2 with bidirectional pipe communication.	Post v1 release